

A PROJECT REPORT

on

**Comparative Analysis of Extractive Text Summarization Methods
on Long Documents.**

Submitted by

VIVEK PAL

Roll No.: CS24012

Seat No.: 1294015

in partial fulfillment for the award of the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE Part II (SEM IV)

under the guidance of

Prof. Vipul Saluja

Department of Computer Science



**R. D. National College of Arts, Science and Commerce
Bandra, Mumbai – 400050.
(2025 - 2026)**

ABSTRACT

Extractive text summarization on long documents remains an underexplored area, with most existing studies evaluated on short to medium length documents that do not adequately represent real-world long-form text. Summarizing long documents remains challenging due to increased information density, semantic diversity, and redundancy. This research presents a comparative analysis of three extractive text summarization methods - TF-IDF, LexRank, and a cluster-based extractive summarization approach using Sentence-BERT (SBERT) embeddings.

WikiHow Dataset was used for its long and structured articles. It is also resistant to Lead-3 bias commonly observed in standard news-based datasets like CNN/DailyMail. The dataset was filtered and articles with minimum 800 words were considered long document for this study. A sample of 500 articles was used for evaluation. Performance was assessed using ROUGE-1, ROUGE-2, and ROUGE-L metrics, alongside coverage and redundancy measures. In addition, document length analysis was done using scatter plots, regression plots, and correlation matrix.

Results indicate that the SBERT-based method achieved the highest ROUGE-1 (0.309) and ROUGE-L (0.166) scores, while LexRank achieved the highest ROUGE-2 score (0.078). TF-IDF demonstrated the highest lexical coverage (0.532), whereas SBERT produced the least redundant summaries (0.325) and LexRank produced the most redundant summary (0.442). SBERT faces a trade-off between redundancy and coverage, while it produced least redundant summaries it had the lowest lexical coverage.

Correlation analysis revealed negligible relationship between document length and ROUGE scores within the 800+ word range, suggesting no evidence of performance degradation with increasing document length in the WikiHow Dataset. Overall, SBERT performed best of the three methods but only had modest ROUGE score gain over LexRank suggesting that Long document summarization remains challenging.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Introduction	02
1.2 Objective	03
1.3 Scope/Significance of the work	03

Chapter 2: Literature Survey

2.1 Classical Extractive Approaches	05
2.2 Graph-Based Methods	05
2.3 Neural / Transformer-Based Extractive Methods	06
2.4 Limitation	07

Chapter 3: Proposed Methodology

3.1 Dataset	09
3.2 Summarization Methods / Models	
3.2.1 TFIDF	10
3.2.2 LexRank	11
3.2.3 SBERT	13
3.3 Evaluation Metrics	14

Chapter 4: Experimental Setup

Chapter 5: Implementation Details

5.1 Data Pipeline	19
5.2 TF-IDF Implementation	19
5.3 LexRank Implementation	20
5.4 BERT-based Implementation	20
5.5 Evaluation Pipeline	20

Chapter 6: Result & Analysis of Result

6.1 Qualitative Example	21
6.2 ROUGE Evaluation Results	22

6.3 Coverage	22
6.4 Redundancy	23
6.5 Visualization	24
 Chapter 7: Conclusion	 33
 Chapter 8: Future Enhancement	 34
 References	 35
 Appendix – Program Code	 38

Title: Comparative Analysis of Extractive Text Summarization Methods on Long Documents.

1. INTRODUCTION:

Text summarization is one of the most critical and challenging tasks in Natural Language Processing (NLP). It is defined as the process of distilling the most important information from a source to produce an abridged version for a particular user and task [1]. Text summarization aims to condense large volumes of text into shorter versions while preserving the most important information, thereby enabling efficient information access and decision-making. Automatic Text summarization is the technique to identify the most useful and necessary information in a text [2].

Early efforts in building automatic text summarization (ATS) systems date back to the 1950s. Subsequently, unsupervised feature-based systems emerged with advances in statistical machine learning during the 1990s and early 2000s. In the 2010s, the focus of summarization research shifted toward training deep learning frameworks in a supervised manner, leveraging the availability of large-scale training data. More recently, the advent of self-supervised pre-trained language models (PLMs) such as Bidirectional Encoder Representations from Transformers (BERT) and T5 has significantly enhanced summarization performance. This progression culminates in the current era dominated by large language models (LLMs). Looking back at the development history of summarization approaches, we can generally categorize it into four stages based on the underlying paradigms: the statistical stage, the deep learning stage, the PLM fine-tuning stage, and the current LLM stage.[3][4][5]

In recent years, the rapid growth of digital textual data across domains such as news media, online encyclopedias, academic publications, and reports has created a strong need for automatic text summarization. Automatic Text summarization is the technique to identify the most useful and necessary information in a text [6]. There are broadly two types of automatic text summarization techniques: 1. Extractive 2. Abstractive. Extractive summarization generates summaries by selecting and combining important sentences as it is in the source document. It doesn't generate any new sentence that is not already present in the source document. On the contrary, Abstractive summarization produces new sentences that paraphrase the content of original document. Extractive summarization techniques are preferred over Abstractive techniques due to their simplicity, interpretability, and lower computational requirement and is widely used in practical applications.

Despite significant progress in extractive summarization research, most existing approaches have been evaluated primarily on short or medium-length documents. However, real-world

textual data often consists of long documents containing multiple sections and redundant information. The increased length and structural complexity of such documents make it difficult for extractive methods to effectively identify the overall important sentences in the document while avoiding redundancy.

Long-document summarization presents unique challenges, including effective content coverage, redundancy reduction, and sentence relevance estimation across extended contexts like a long document. Understanding how different extractive summarization techniques perform under long-document conditions is essential for assessing their suitability for real-world applications which involves lengthy textual content.[3]

In this context, the present study focuses on the evaluation of extractive text summarization methods for long documents. By analyzing and comparing different extractive methods, this research aims to provide insights into their strengths and limitations when applied to long-form textual data.

1.2 OBJECTIVE:

The Research Proposal aims to study how extractive summarization methods perform on long documents. The Detailed Objectives are as follows:

1. To compare different extractive summarization approaches
2. To evaluate their performance using standard evaluation metrics
3. To analyze limitations such as redundancy and content coverage

1.3 SCOPE / SIGNIFICANCE OF THE WORK:

The scope of this research is specifically bounded by the evaluation of extractive summarization techniques on long documents. Unlike abstractive methods that paraphrase content, this study focuses solely on methods that select and combine existing sentences from source documents. The study utilizes English-language long-form documents derived from the WikiHow dataset. It focuses on articles where important information is distributed across the entire text rather than concentrated at the beginning. The study focuses on comparing representative extractive approaches, including classical statistical methods, graph-based

sentence ranking techniques, and transformer-based extractive model. The work prioritizes quantitative analyses using standard ROUGE scores alongside mathematical measurements of redundancy and coverage.

This research addresses a critical gap in Natural Language Processing: the degradation of summary quality as document length increases. This work is significant as most existing summarization studies predominantly evaluate models on short or medium-length documents, which do not adequately reflect real-world scenarios involving long-form text such as reports, instructional guides, and structured articles. While news-based datasets often rely on the “Lead-3” baseline due to their “inverted pyramid” writing style, this study uses WikiHow to provide a more rigorous test of a model's ability to achieve high content coverage across extended contexts. By focusing on long documents, this research provides insights into the limitations and behavior of extractive summarization methods beyond standard benchmark settings.

The findings of this study help in understanding the trade-offs between coverage and redundancy in extractive summarization for long documents. This study does not propose a novel architecture but rather conducts a comparative and analytical evaluation of existing methods under long-document conditions.

2. LITERATURE SURVEY:

Automatic text summarization has been an active research area for several decades. Throughout the literature, there have been different approaches applied to generate an extractive summary. Some of the approaches have been discussed below:

2.1 Classical Extractive Approaches

In the year 1958 paper titled “The Automatic Creation of Literature Abstracts”, H. P. Luhn laid the foundation of frequency-based summarization. This established the "Frequency-based" paradigm. He hypothesized that the most important information in a document is contained in sentences with a high concentration of "significant" words. He calculated the importance of a sentence based on the proximity of these significant words. Luhn’s method is highly efficient for long documents because it doesn't require complex training, but it fails on long documents because it only counts repetition and lacks semantic understanding.[7][8]

Classical extractive summarization approaches commonly employ features such as word frequency, sentence position, term frequency–inverse document frequency (TF-IDF), and cue words to rank sentences according to their importance. These methods are computationally efficient and have shown reasonable performance on short and moderately sized documents.

However, multiple studies indicate that such techniques struggle to capture semantic relationships and often lead to redundancy when applied to longer documents. This limits their effectiveness in real-world scenarios involving extended text [6,9].

2.2 Graph-Based Methods

The year 2004 was a turning point for graph-based NLP. TextRank was introduced in a 2004 paper titled “TextRank: Bringing Order into Texts” by Mihalcea & Tarau and LexRank was introduced in the 2004 paper titled “LexRank: Graph-based Lexical Centrality as Salience in Text Summarization” by Güneş Erkan and Dragomir R. Radev. Both methods treat the document as a network, but differently calculate the strength of connection.

TextRank applies the PageRank algorithm to text. A sentence is “important” if it is linked to other “important” sentences. It is calculated by the number of shared tokens between two sentences, normalized by their lengths. LexRank introduces “Lexical Centrality”, It uses the Cosine Similarity of TF-IDF vectors to determine edge weights. [10][11]

Graph-based extractive summarization methods address some of the limitation posed by statistical method by modelling documents as graphs, where sentences are represented as nodes and edges denote similarity relationships. It models sentences as nodes in a similarity graph and apply ranking or centrality measures to identify salient content, demonstrating effectiveness on short and moderately sized documents.

Techniques such as TextRank, LexRank, and their variants apply centrality measures to identify important sentences. These methods improve coherence and reduce local redundancy compared to purely statistical approaches.

However, several studies report that graph-based methods face scalability challenges and reduced effectiveness when applied to long document. This is particularly due to dense similarity graphs and difficulty in identifying globally relevant content in long documents. [12][13][14][15]

2.3 Neural / Transformer-Based Extractive Methods

There have been different approaches applied to generate an extractive summary using Neural/Transformer based methods. In 2016, Cheng and Lapata proposed an attentional encoder–decoder architecture for an extractive single-document summarization trained on the CNN/DailyMail corpus. Their model is based on an encoder–decoder approach, where the encoder learns the representation of sentences and documents while the decoder classifies each sentence based on the encoder’s representation using an attention mechanism. In 2019, Joshi, Eduardo, Enrique, and Laura proposed SummCoder, an unsupervised framework for extractive text summarization based on deep auto-encoders. In their approach, the extractive summarization problem was defined as a sentence selection problem given a document. The determination of which sentence should be included in the summary was based on three metrics:

- 1.Sentence Content Relevance Metric;
- 2.Sentence Novelty Metric;
- 3.Sentence Position Relevance Metric.

In 2019, Yang Liu proposed a fine-tuned BERT summarization approach called BERT-Sum. BERT is a stack of pre-trained transformer encoders that can better understand the textual data through attention mechanisms. [16] [17]

Recent advancements in extractive summarization have increasingly focused on neural and transformer-based models. Deep learning approaches frame extractive summarization as a sentence-level classification or ranking problem, leveraging contextual representations generated by neural encoders. Transformer-based models, particularly those derived from BERT, have demonstrated strong capability in capturing contextual and semantic information across sentences. Several studies report improved ROUGE scores using BERT-based extractive frameworks compared to traditional methods. [18]

However, these models often require substantial computational resources and are frequently evaluated on datasets consisting of short or medium-length documents, limiting insights into their behavior on long-document summarization tasks.[19][20]

2.4 Limitations in Long-Document Summarization

Despite the progress achieved by classical, graph-based, and transformer-based extractive summarization methods, existing literature identifies challenges in handling long documents. Current Large Language Model based models struggle to handle long-form documents due to input token constraints. When input texts exceed these constraints, truncation can lead to the loss of essential information degrading the summary quality. [3]

The CNN/DailyMail dataset is the most widely-used dataset for summarization. It has been used as the predominant benchmark for assessing summary quality and comparing the efficacy of summarization methodologies in recent years. It contains news articles and associated highlights as summaries written by journalists from CNN and the DailyMail. [21] It is a short news summarization dataset consisting of small to medium length articles. These articles follow an “inverted pyramid” style i.e. most important information comes first. Because of this, even the most advanced AI often just learns to “pick the first 3 sentences”. This is called “Lead3 Baseline”. WikiHow is a diverse dataset extracted from online knowledge base. Each article consists of a number of steps focusing on How-to instructions. Summarization systems perform a lot better on CNN/Daily mail compared to the WikiHow dataset with lead-3 outperforming other baselines due to the news inverted pyramid writing style. On the other hand, the **poor performance of lead-3 on WikiHow** shows the different writing styles in its articles.[22][23]

This suggest WikiHow is a superior test for long-document extraction. Unlike news, the “important” parts of a “How-to” guide are spread across the whole document (Step 1, Step 2,

Step 5, etc.) this forces the model to look at the *entire* document to achieve high content coverage. If a model only looks at the first 512 tokens (the Lead Bias), it will miss the final steps of the instruction, leading to a low ROUGE score.

Commonly reported issues include sentence redundancy, inadequate content coverage, and difficulty in modelling global document structure. Recent survey studies emphasize the need for focused evaluations that analyze how extractive summarization techniques perform under long-document conditions.

Existing extractive text summarization techniques have been widely studied and have shown to be effective and computationally efficient for short to medium length documents.

However, some studies have shown that these techniques exhibit weak performance on long documents, due to reasons like redundancy and inadequate content selection. The increased document length introduces challenges like identifying overall important content while still maintaining coherence. Many existing evaluations focus on standard datasets that do not adequately represent long-document scenarios. This gap encourages a comparative evaluation of extractive summarization methods under long-document conditions. Accordingly, this research aims to evaluate and compare different extractive summarization techniques to better understand their limitations in summarizing long document.

3. PROPOSED METHODOLOGY:

This study finds how extractive summarization methods behave on long documents. It utilizes Quantitative Research design approach to focus on extractive summarization of long documents, where existing methods often suffer from redundancy and poor content coverage. The proposed research compares different extractive summarization methods, starting from simple statistical technique to more recent embedding-based approaches and evaluate their performance using standard evaluation metrics to study how they perform on long documents.

3.1 Dataset

This study utilizes long-document textual data derived from WikiHow articles using ‘WikiHow’ dataset [28]. WikiHow dataset introduced by Koupae and Wang (2018) is a large-scale summarization dataset derived from the WikiHow online knowledge base. It is a good dataset for long-document extractive summarization because of its rigid, hierarchical structure. It is built in a way that creates a natural extractive loophole. Every wikiHow article consists of several “Steps”.

The ‘important’ parts of a ‘How-to’ guide are spread across the whole document (Step 1, Step 2, Step 5, etc.). This forces the model to look at the *entire* document to achieve high content coverage. This helps in beating the “Lead-3” Bias known in popular datasets like CNN/DailyMail.

Unlike news-based datasets such as CNN/DailyMail, WikiHow articles distribute important information across the entire document rather than concentrating it at the beginning, making it a more rigorous benchmark for evaluating long-document summarization.

The full dataset contains approximately 215,000 articles, each consisting of a series of instructional steps accompanied by bolded headline summaries. To focus the evaluation on long documents, articles with fewer than 800 words were excluded from consideration. This filtering resulted in approximately 30,000 documents. From this filtered set, a random sample of 500 documents was selected for evaluation, balancing computational feasibility with statistical representativeness.

The dataset consists of three columns: ‘headline’, ‘title’, and ‘text’. The ‘headline’ contains concatenation of all bold lines in the article. It serves as the reference summary for evaluation. The ‘text’ contains concatenation of all the paragraphs in the articles (excluding bold lines) which will be summarized using different summarization models. The generated summaries are then evaluated against reference summaries using standard evaluation metrics to test how well the model performed.

The dataset initially contained 215,365 entries, of these several rows in ‘text’ and ‘headline’ columns had null values. In the text column, each paragraph was separated by commas and there were new-line (\n) characters in between paragraphs to separate the paragraphs. These were noises and needed to be removed. As a preprocessing step, we dropped these null values from the dataset and removed noises in the text column.

WikiHow articles are selected as the primary data source due to their length, structured content, and wide use in summarization research. These documents are ideal to evaluate the performance of extractive summarization methods in handling redundancy and content coverage in long documents.

3.2 Summarization Methods / Models

The methodology evaluates three extractive summarization approaches, including a statistical baseline method, a graph-based sentence ranking method, and a transformer-based sentence embedding approach using pre-trained language model. It analyzes their performance on long textual documents.

3.2.1 TF-IDF

TF-IDF serves as the statistical baseline method for analysing long documents. Term Frequency (TF) measures how frequently a term occurs in a specific document. The formula is:

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of words in document } d}$$

Inverse Document Frequency (IDF) measures how important a term is across the entire corpus. The formula is:

$$\text{IDF}(t) = \log \left(\frac{\text{Total documents}}{\text{Documents containing term } t} \right)$$

The final TF-IDF score for a term 't' is obtained by multiplying its TF and IDF scores:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

A higher score indicates that a word is highly important to the meaning of a particular document. Common words such as 'the' or 'and' receive low scores as they appear across all documents, while domain-specific or contextually significant words receive higher scores.

The TF-IDF based method was implemented using a Scikit-Learn 'TfidfVectorizer' tool, which convert raw text documents into a matrix of numerical features. It calculates Term Frequency (TF) and Inverse Document Frequency (IDF) scores for each word. This highlights words that are unique to specific documents while penalizing common, less informative words. The matrix of numerical features is then summed along columns. This produces a score for each sentence, which is then sorted to select the top 5 sentences in the article. These five sentences with the highest cumulative TF-IDF scores are selected to form the extractive summary.

3.2.2 LexRank

LexRank serves as the graph-based sentence ranking method for analyzing long documents. It is an unsupervised, graph-based algorithm used for extractive text summarization. It identifies the most important sentences in a document by determining their 'lexical centrality'.

Sentences that have the highest lexical similarity (word overlap) with many other sentences are considered more central and are ranked higher. It is heavily inspired by Google's PageRank algorithm, which ranks websites based on how many other sites link to them.

In the LexRank algorithm, the approach is that sentences 'recommend' other similar sentences. If one sentence is very similar to many others, it will likely be a sentence of great importance. The importance of this sentence also stems from the importance of the sentences

‘recommending’ it. Thus, to get ranked highly and placed in a summary, a sentence must be similar to many sentences that are in turn also similar to many other sentences.

LexRank constructs a similarity graph in which each ‘node’ represents a single sentence in the document. The graph looks like a map of interconnected nodes (sentences) where the lines between them (edges) vary in thickness depending the similarity score (cosine similarity on TF-IDF vectors). The similarity score dictates the weight of the connecting edge, stronger relationships are drawn with thicker lines or higher weight values, while weak relationships look like thin, faint lines.

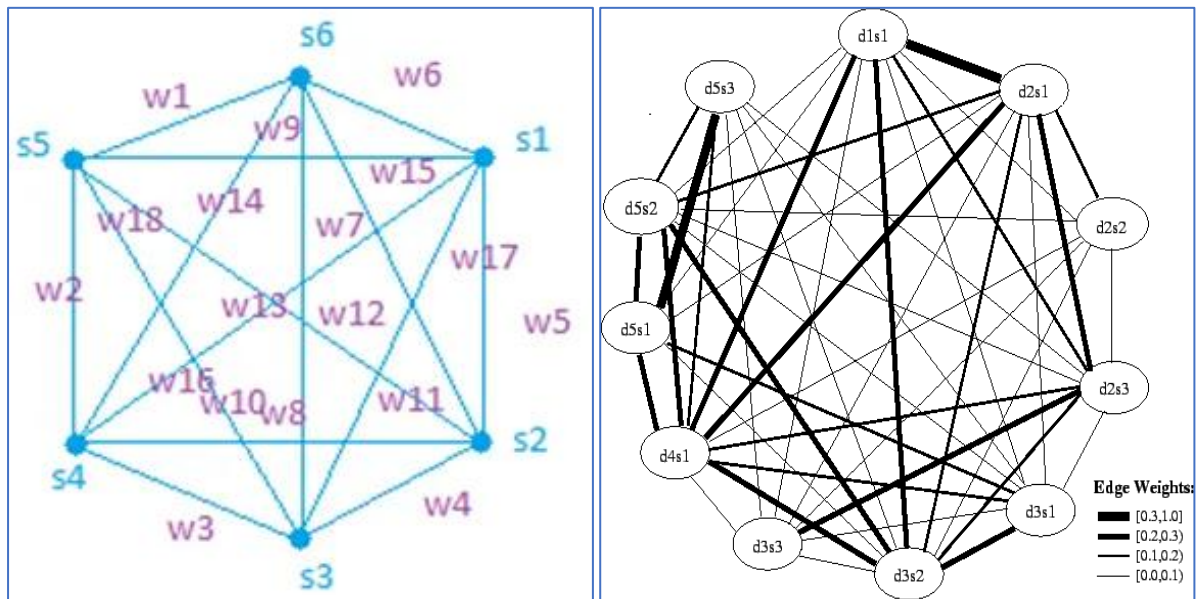


Figure 3.1: Sentence similarity graph examples used in LexRank: (a) weighted sentence similarity graph [10]; (b) LexRank sentence similarity graph [11].

The algorithm uses cosine similarity on TF-IDF vectors to measure how similar a sentence is to all other sentences in the document. Cosine similarity measures how closely aligned two vectors are in a multi-dimensional space. It calculates the cosine of the angle between them, ignoring their magnitude. Instead of measuring the physical distance between data points, cosine similarity focuses on orientation. For instance, if two documents cover the same topic, their TF-IDF vectors will point in similar directions, yielding a high similarity score regardless of document length. Scores range from -1 (exact opposite) to 1 (identical), where 0 means the vectors are completely independent. Cosine similarity is calculated by dividing the dot product of two vectors $\vec{A}$ and $\vec{B}$ by the product of their magnitudes:

$$\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

where:

$\vec{A} \cdot \vec{B}$ = Dot product of vectors $\vec{A}$ and $\vec{B}$

$\|\vec{A}\|$ = Magnitude (Euclidean length) of vector $\vec{A}$

$\|\vec{B}\|$ = Magnitude (Euclidean length) of vector $\vec{B}$

LexRank determines the most important sentences by calculating ‘Eigenvector Centrality’. Sentences that are highly similar to many other sentences that are also highly similar to the rest of the document receive higher centrality scores. The sentences with the highest LexRank scores are selected for the summary.

3.2.3 SBERT

SBERT serves as the transformer-based sentence embedding approach for analysing long documents. Transformer is a type of Neural Network that processes entire sequence of data in parallel. BERT (Bidirectional Encoder Representation from Transformer) is an open-source language model for NLP developed by Google. It is designed for language understanding and does not generate any text. It reads data bidirectionally i.e. from left to right as well as right to left simultaneously. This gives the model greater understanding of the word context. For example, the word 'bank' is interpreted differently in the context of 'river bank' versus 'money bank'.

SBERT is a modified version of BERT. It is designed for generating meaningful sentence embeddings. Regular BERT is slow and not very accurate when used to compare two sentences for similarity, as both sentences must be fed together into the model simultaneously. SBERT addresses this limitation by training BERT using a siamese network structure — meaning it learns to produce a single vector (embedding) for each sentence that captures its meaning. Sentences can then be compared by computing cosine similarity between their vectors. SBERT belongs in ‘sentence-transformers’ library maintained by Hugging-face. ‘sentence-transformers’ is specialized for creating sentence/paragraph level embeddings. Here, semantically similar texts are close together in vector space. ‘sentence-transformers’ is built on top of ‘transformers’ library.

SBERT have token limit of 256-512 token limit. So if the sentence is very long and goes beyond the limit, it gets truncated automatically. To mitigate this, we segment the text to be summarized into individual sentences by tokenizing it and then pass individual sentences one by one to SBERT. This makes sure the token limit is not reached and text is not getting truncated.

SBERT converts each sentence into vector embeddings. All the vector embeddings are then grouped together into 5 clusters using K-Means. K-Means is an unsupervised machine learning algorithm which groups sentences into N clusters and finds the geometric center of each cluster. The center is a N-dimensional vector - the "average" position of all sentences in that cluster. Finally, sentences which are closest to each cluster centre are selected to form the extractive summary. This is called as ‘Cluster-based extractive summarization using SBERT embeddings.’

3.3 Evaluation Metrics

Generating a summary does not have an absolute correct answer. Each summary generated by a human reader differs from another human reader based on what he or she perceives as important information to include. In text summarization, an evaluation metric often used is Recall-Oriented Understudy for Gisting Evaluation, or in short, ROUGE. It measures the overlap between the reference summary and the generated summary. There are different variants of ROUGE metric depending on how many overlapped words are measured. These variants include ROUGE-1, ROUGE-2, and ROUGE-L. [18]

ROUGE-1 measures overlap of individual words appearing in generated summary and reference summary (headline). It shows how many single words match in the reference summary vs generated summary. ROUGE-2 measures overlap of word-pairs (bi-grams) appearing in generated summary and reference summary. ROUGE-2 is stricter than ROUGE-1. ROUGE-1 evaluates how much of the necessary vocabulary or content was captured. ROUGE-2 evaluates fluency and word order, checking if the phrases make grammatical sense. ROUGE-L measures longest common sub-sequence of words (LCS). It evaluates sentence structure. It determines how well summary covers sequential flow of original document (not just word-overlap).

In ROUGE evaluation, each ROUGE score consists of recall, precision, and the F-measure (F1-score) which determine exactly how well a generated text matches a human reference text. Recall measures how much of the reference summary text was successfully captured by the generated summary text.

Its formula is given as:

$$\text{Recall} = \frac{\text{Number of overlapping n-grams}}{\text{Total n-grams in the Reference Summary}}$$

High Recall means the model successfully included most of the facts from the reference text.

Precision measures how much of the generated summary text was actually relevant and present in reference summary text. Its formula is given as:

$$\text{Precision} = \frac{\text{Number of overlapping n-grams}}{\text{Total n-grams in the Generated Summary}}$$

High Precision means the model was concise and did not output unnecessary words.

The F-measure is the harmonic mean of precision and recall. Its formula is given as:

$$\text{F-Measure} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

It ranges from 0 (worst) to 1 (best). Harmonic mean is taken instead of normal mean to punish extreme cases where recall is very high and precision is very low and vice-versa.

Content coverage measures the fraction of words in the reference summary that appear in the generated summary. Content coverage is calculated using ROUGE Recall score. A higher ROUGE-1 recall score indicates greater coverage. ROUGE-1 recall is used as a proxy measure for measuring reference summary lexical coverage.

Redundancy measures the repetitiveness of the generated summary. It occurs when a model selects multiple sentences that convey the exact same information or facts. Extractive summarization is highly prone to this issue. Redundancy is calculated using cosine similarity. To measure redundancy, each sentence in the generated summary is first converted into a numerical vector (embedding) using the SBERT. The semantic similarity between every pair of summary sentences is then computed using Cosine Similarity. The average of

these pairwise similarity scores serve as the redundancy measure, where a higher value indicates greater repetitiveness in the generated summary.

A higher redundancy score indicates that the selected sentences are more semantically similar i.e. redundant, suggesting greater repetition of information. Conversely, a lower redundancy score indicates that the summary contains more diverse information with less repetition. The average redundancy score over all generated summaries was used to compare the redundancy characteristics of the different extractive summarization methods.

The summarization model outputs are evaluated using standard ROUGE metrics like ROUGE-1, ROUGE-2, ROUGE-L (F-measure). Along with that, redundancy and content coverage are also be analyzed to check the impact that the long-document length will have on summary quality.

4. EXPERIMENTAL SETUP:

This chapter describes the experimental setup used to conduct the comparative evaluation of the three extractive summarization methods. All experiments were performed on a local machine running Windows 11. The hardware configuration, software requirements, and libraries used are detailed below.

4.1 Hardware Configuration

The experiments were conducted on a system with the following hardware configuration:

Component	Specification
Processor	13th Gen Intel Core i5-13500H
RAM	16 GB
Storage	1 TB SSD
Operating System	Windows 11 Home
Graphics Card	Intel(R) Iris(R) Xe Graphics

4.2 Software Requirements

Python 3.11 was installed and managed using the Python Install Manager. A dedicated virtual environment was created to isolate project dependencies and ensure reproducibility of the experimental setup.

The experiments were conducted using the following software components:

Software	Version
Python	3.11
Visual Studio Code	Latest
Jupyter Notebook	Latest
Python Install Manager	26.3.240.0

4.3 Libraries Used

The experiments utilized the following libraries for data processing, model implementation, evaluation and result visualization:

Library	Purpose
pandas	Data loading and preprocessing
scikit-learn	TF-IDF vectorization and K-Means clustering
sumy	LexRank summarization
sentence-transformers	BERT sentence embeddings
rouge-score	ROUGE metric evaluation
matplotlib / seaborn	Result visualization

5. IMPLEMENTATION DETAILS:

The project was implemented in Python 3.11 using Visual Studio Code as the development environment. The implementation was organized across four Jupyter notebooks — one for each summarization method and a separate notebook for visualization.

The key libraries used include pandas for data handling, scikit-learn for TF-IDF vectorization and K-Means clustering, sumy for LexRank summarization, sentence-transformers for BERT-based sentence embeddings, and rouge-score for ROUGE evaluation.

5.1 Data Pipeline

The WikiHow dataset was downloaded from the dataset's official GitHub repository and loaded from a CSV file using pandas read_csv function. The Dataset contains approximately 215,365 articles. Articles with null values in the 'text' or 'headline' columns were removed, and noise such as newline characters was cleaned from the text column. Articles with fewer than 800 words were excluded, producing a filtered set of approximately 30,000 long documents. A random sample of 500 documents was then selected for evaluation. To avoid reloading and reprocessing the dataset across notebooks, the filtered dataframe was serialized and saved using Python's pickle library. Pickle file makes sure that Data Type of columns don't change upon saving and loading it in new notebook.

5.2 TF-IDF Implementation

The TF-IDF summarizer was implemented using the TfidfVectorizer class from the scikit-learn library. Each document was split into individual sentences using NLTK's sentence tokenizer – 'punkt', and the vectorizer was applied to convert all sentences into a TF-IDF feature matrix. Sentence scores were computed by summing TF-IDF values across all terms in each sentence. The five sentences with the highest scores were selected as the extractive summary.

5.3 LexRank Implementation

LexRank was implemented using the sumy library. Each document was parsed and the LexRank algorithm was applied to construct a sentence similarity graph using cosine similarity on TF-IDF vectors. Eigenvector centrality scores were computed for each sentence node, and the five highest-ranked sentences were selected to form the summary.

5.4 BERT-based Implementation

The BERT-based summarizer was implemented using the ‘all-MiniLM-L6-v2’ model from the sentence-transformers library. This is a smaller and faster version of SBERT. Each sentence in the document was encoded into a fixed-dimensional vector embedding. The resulting embeddings were then clustered into five groups using the K-Means algorithm from scikit-learn. The sentence whose embedding was closest to each cluster centroid was selected as a representative summary sentence, yielding a five-sentence extractive summary.

5.5 Evaluation Pipeline

Generated summaries from all three methods were evaluated against the reference summaries using the rouge-score library, computing ROUGE-1, ROUGE-2, and ROUGE-L F-scores. Content coverage was measured using ROUGE-1 recall. Redundancy was measured by encoding each summary sentence using the sentence-transformers model and computing pairwise cosine similarity between all sentence pairs within the summary. The mean of these pairwise scores served as the redundancy value for each document. Results were aggregated across all 500 documents and visualized using matplotlib and seaborn.

6. RESULT & ANALYSIS OF RESULT:

6.1 Qualitative Example

The following table presents a Qualitative example of generated summaries of each of the summarization methods:

Reference Summary	'Meditate. Watch a movie. Spend time with friends. Go for a long drive. Read. Calm your mind before bed.'
TF-IDF Summary	When you watch a movie, try to clear your mind as much as possible and think about what the characters are doing and saying instead of what you're going to do or say after the movie. If you have a busy schedule, choose a board game night or go out to see a comedy with your friends, instead of going to a crowded bar where you won't have as much of a chance to laugh. You may feel overpowered by nasty traffic, or annoyed by how aggressively other people drive during the day, but if you take to the roads at night, you'll feel calmer and more empowered. If you're leaving a party after hours of laughing, sharing your feelings, and listening to your friends, taking a 20-minute solo drive before going home can help you wind down. If you feel so stressed out that you can't focus on what you're reading, take a break to meditate, or whisper the words aloud until you absorb their meaning.
LexRank Summary	"Going out to the movies or watching a movie on television can help you escape into another universe and to take your mind off of your own problems. When you watch a movie, try to clear your mind as much as possible and think about what the characters are doing and saying instead of what you're going to do or say after the movie. If you have a busy schedule, choose a board game night or go out to see a comedy with your friends, instead of going to a crowded bar where you won't have as much of a chance to laugh. If you love to drive, then going for a long drive late at night can help you relax and feel more in control of your life. Reading will not only improve your knowledge, but it will allow you to rest your body and quiet your mind as you focus on the material in front of you."
BERT Summary	"Be aware of what you can't control. Spending time with friends is a great way to relax. If you're leaving a party after hours of laughing, sharing your feelings, and listening to your friends, taking a 20-minute solo drive before going home can help you wind down. Reading is a fantastic way to relax, especially before bed. Light it with gentle lamplight or candles."

Table 6.1: Summary Comparison Table (Qualitative example).

6.2 ROUGE Evaluation Results

The following table presents the average ROUGE-1, ROUGE-2, and ROUGE-L F-scores, Coverage and Redundancy scores obtained by each summarization method across the 500 sampled WikiHow documents:

Method	ROUGE-1	ROUGE-2	ROUGE-L	Coverage	Redundancy
TF-IDF	0.250	0.070	0.136	0.532	0.399
LexRank	0.278	0.078	0.152	0.487	0.442
BERT	0.309	0.072	0.166	0.370	0.325

Table 6.2: Average evaluation scores across 500 WikiHow documents

The Cluster-based extractive summarizer using SBERT embeddings achieved the highest ROUGE-1 (0.309) and ROUGE-L (0.166) scores, indicating superior capability in identifying semantically important sentences from long documents.

LexRank outperformed TF-IDF across all evaluation metrics by exploiting inter-sentence relationships. It also achieved the highest ROUGE-2 score (0.078), suggesting stronger bigram overlap than BERT. ROUGE-2 measures exact bigram overlap, which may favor extractive methods that choose sentences resembling the reference summary wording.

TF-IDF showed the weakest performance due to its reliance on term frequency statistics without considering semantic meaning or relationships between sentences.

Despite the improvements offered by SBERT embeddings, the relatively modest gains over LexRank suggests that **long-document extractive summarization such as of WikiHow articles remains a challenging task.**

6.3 Coverage

TF-IDF achieved the highest coverage score of 0.532, indicating that its selected sentences contained a greater proportion of reference summary words. LexRank followed with a coverage score of 0.487, while SBERT recorded the lowest coverage of 0.370. This suggests that SBERT's clustering-based selection, while producing more coherent summaries, may miss certain reference words that frequency-based methods capture.

Coverage for SBERT is the worst of all the three methods, this is primarily because of how coverage is calculated. Coverage is based on ROUGE-1 recall score. ROUGE-1 is purely a lexical metric - it counts exact, overlapping word unigrams. It just measures number of words in reference summary that appear in generated summary. So, the coverage score is directly proportional to number of words in generated summary that appear in reference summary.

LexRank and SBERT take into account the semantic meaning of the sentences and not just word frequency, so it may contain words that semantically is suitable but since it's not the exact same word, ROUGE punishes it and this results in lower content coverage score.

SBERT's K-Means clustering selects semantically diverse sentences, but those sentences use different words to express the same ideas as the reference. The reference headline is very short and specific ("Meditate. Watch a movie.") while SBERT picks full sentences that talk about the same topics but with different words. ROUGE-based coverage does not fairly represent SBERT's actual semantic coverage.

SBERT exhibits lower ROUGE-1 recall despite achieving the highest overall ROUGE scores, suggesting that semantic sentence selection does not always maximize lexical overlap with reference summaries.

6.4 Redundancy

SBERT produced the least redundant summaries with a score of 0.325, while LexRank exhibited the highest redundancy at 0.442. TF-IDF fell between the two with a redundancy score of 0.399.

This indicates that SBERT's K-Means clustering approach encourages diversity. If two sentences are very similar, they'll often belong to the same cluster, and only one gets selected.

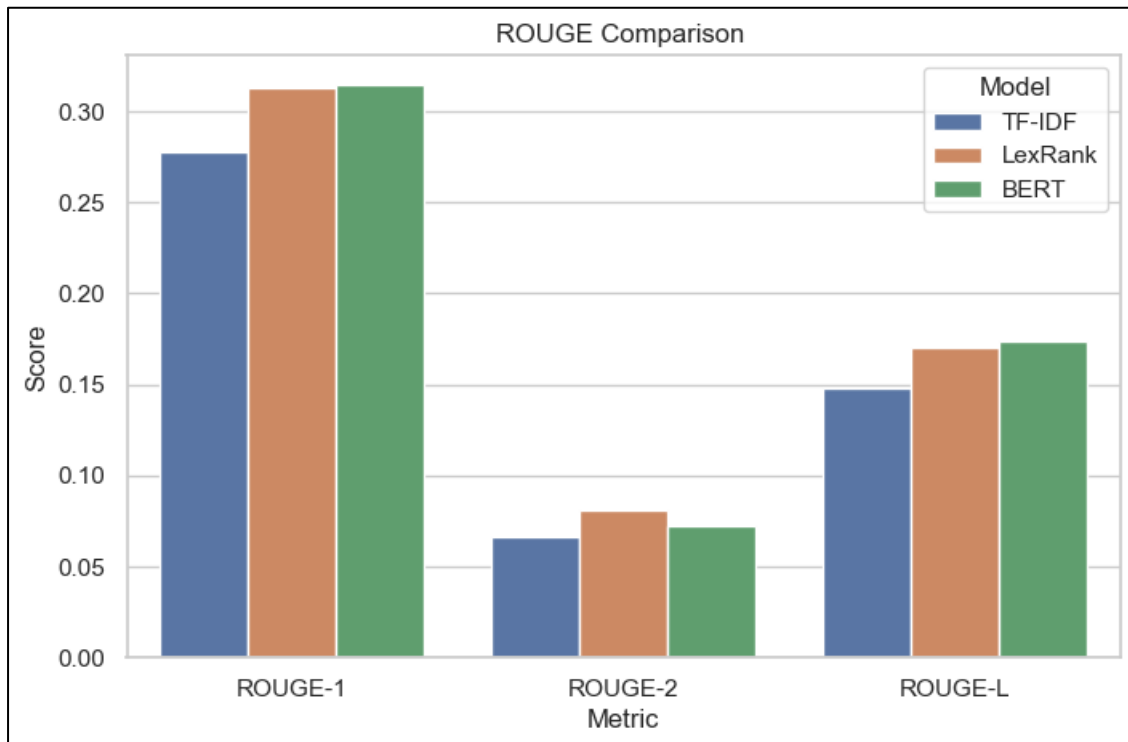
LexRank has the highest redundancy. It selects central sentences that are highly connected to many other similar sentences. This results in higher redundancy.

TF-IDF is in between the two, it selects sentences containing high-scoring words. Those sentences may overlap conceptually, but not as strongly as LexRank's graph based centrality based sentence selection.

6.5 Visualization:

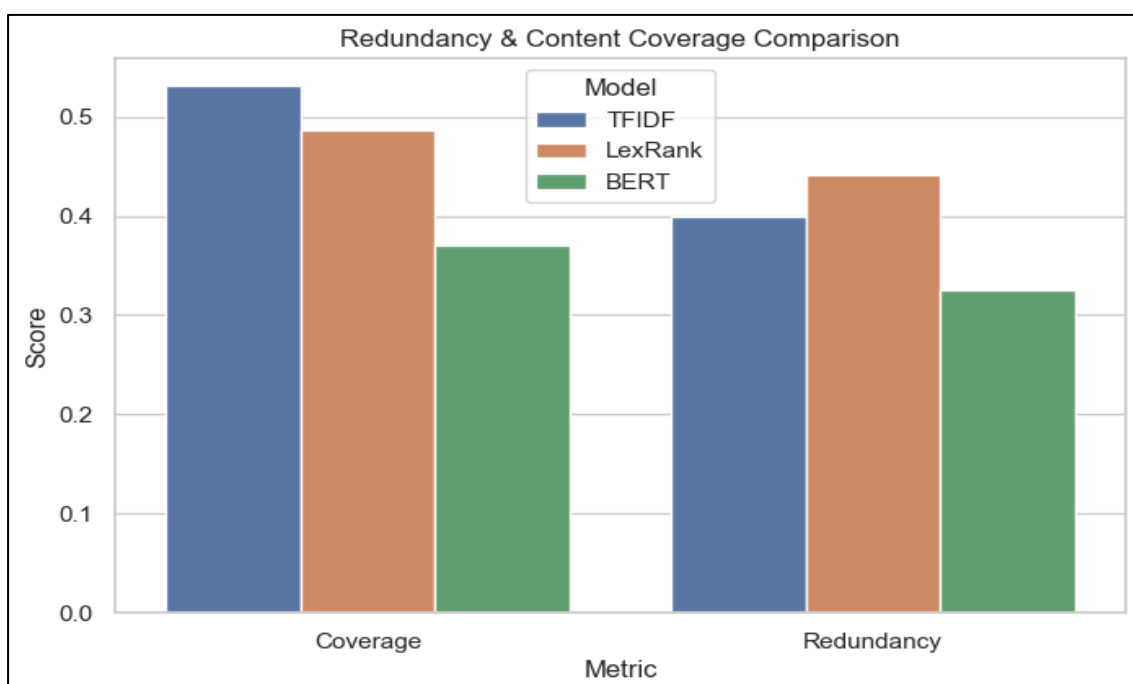
6.5.1 Grouped bar chart - ROUGE-1, ROUGE-2, ROUGE-L

This shows ROUGE score comparison side by side for all three models.



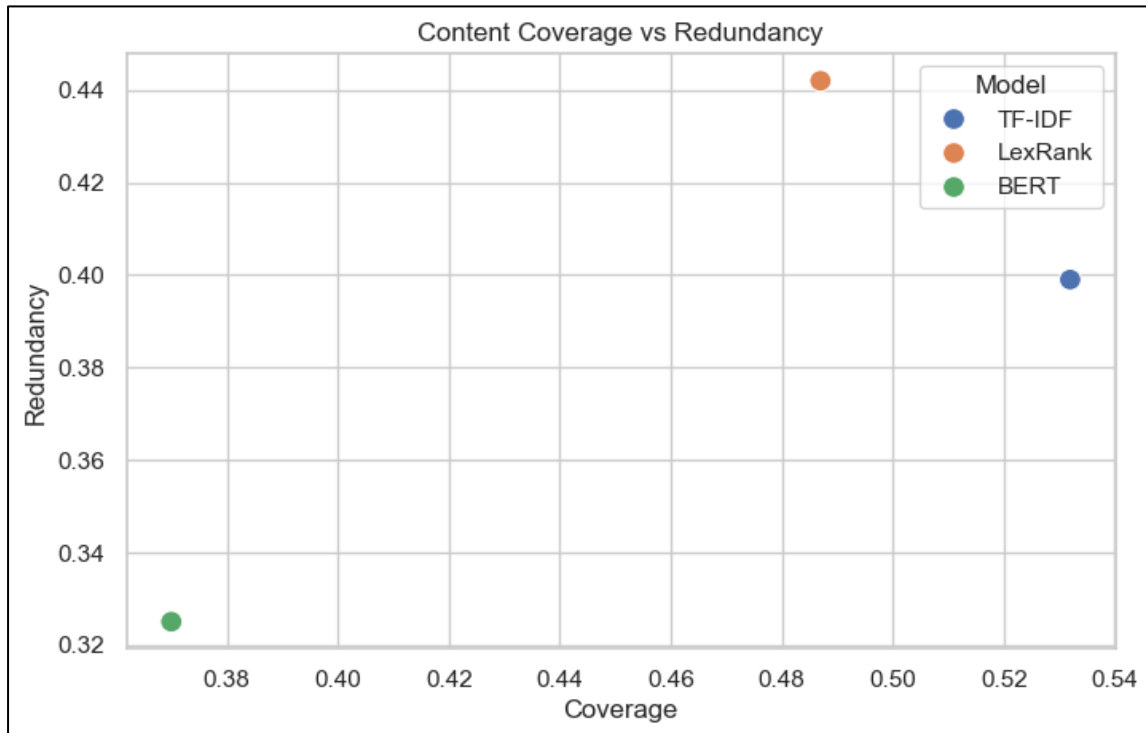
6.5.2 Grouped bar chart - Coverage and Redundancy

This shows Coverage and Redundancy score comparison side by side for all three models.



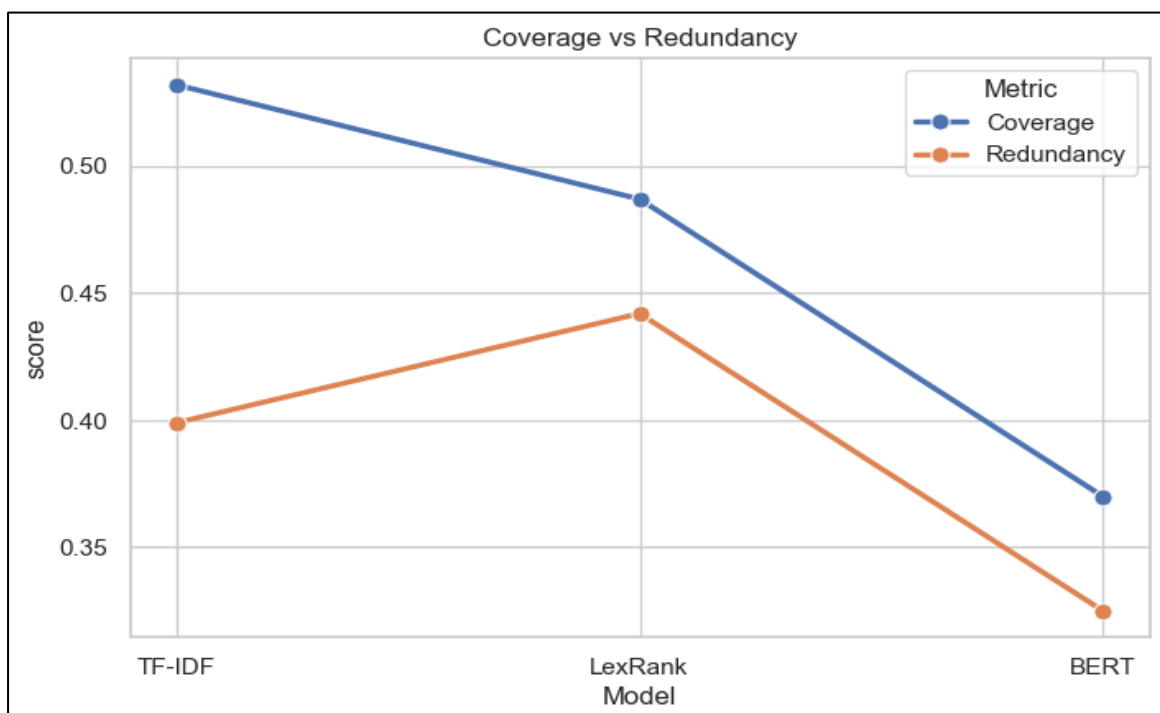
6.5.3 Scatter plot - Coverage vs Redundancy

This shows the trade-off between coverage and redundancy for all three models. Ideal model is high coverage, low redundancy (bottom right).



6.5.4 Line Plot - Coverage vs Redundancy

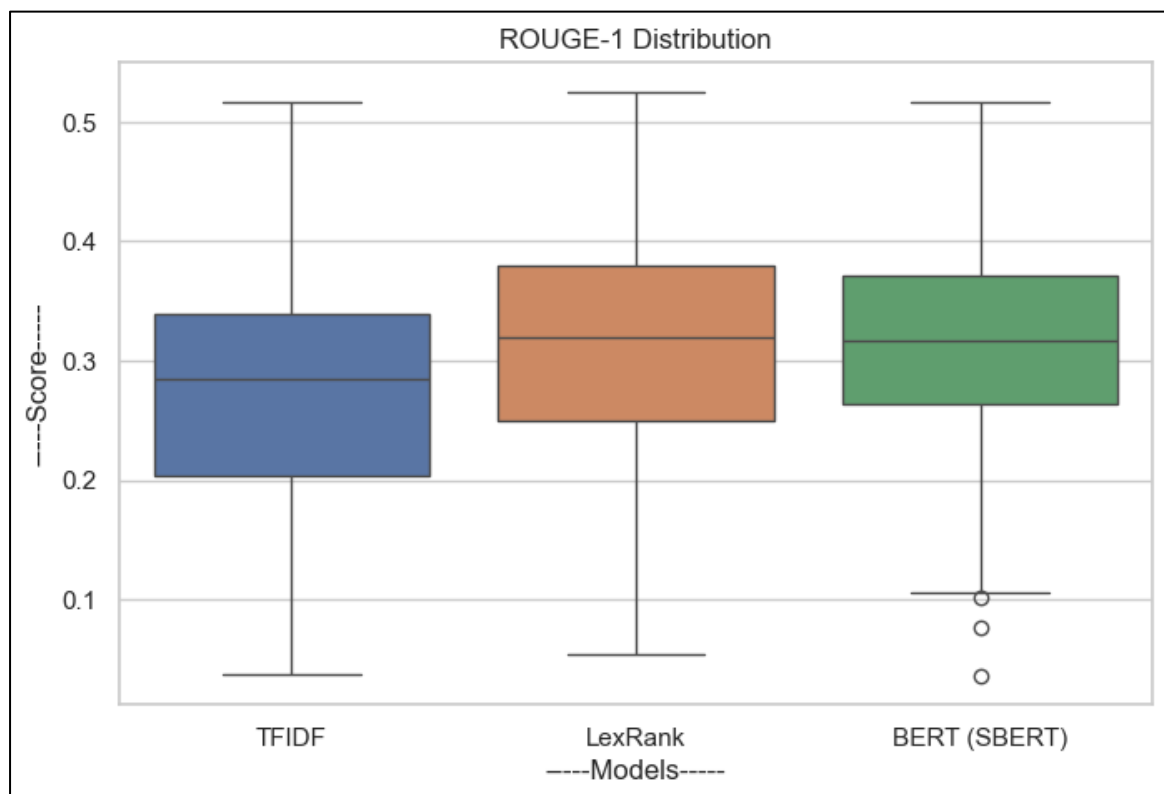
This shows Coverage and Redundancy score as a line plot.



6.5.5 Box plot - Distribution of ROUGE-1

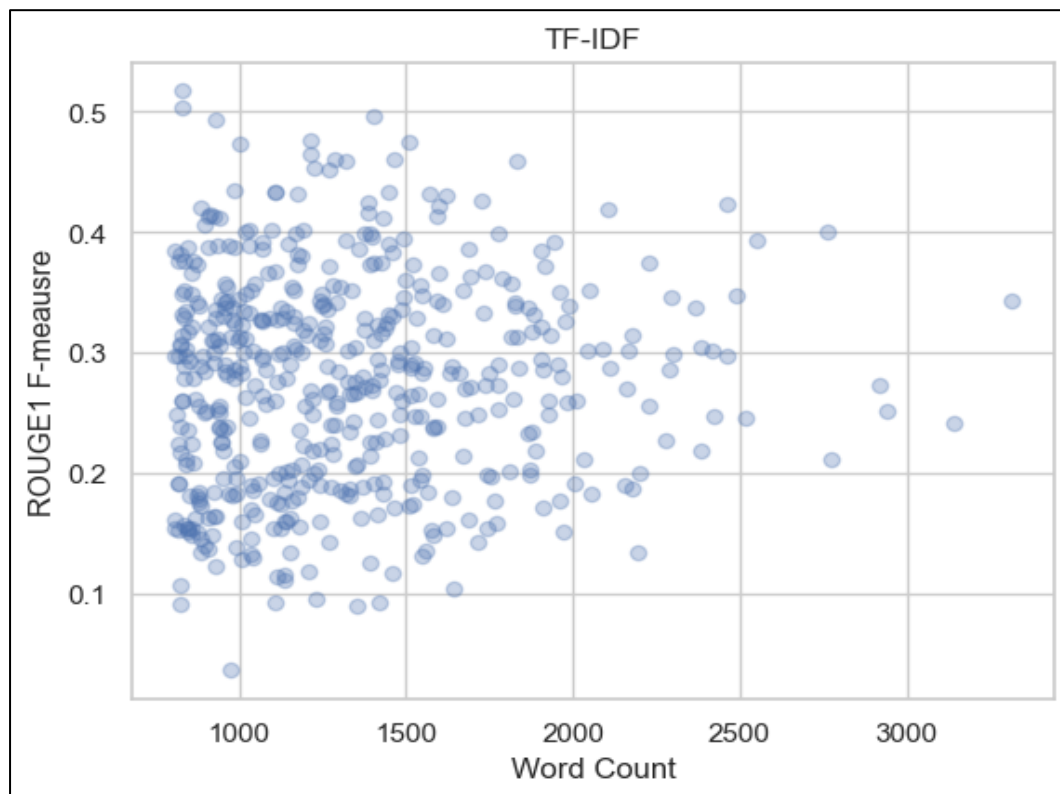
This shows distribution of ROUGE-1 scores across 500 articles for each model. TF-IDF has the lowest median and widest spread indicating most inconsistent performance. LexRank and BERT have similar medians but BERT shows a notably smaller interquartile range, suggesting more consistent performance across documents.

However, BERT exhibits a few low-scoring outliers, indicating occasional failure cases on certain articles. All three models reach similar upper ranges around 0.5.

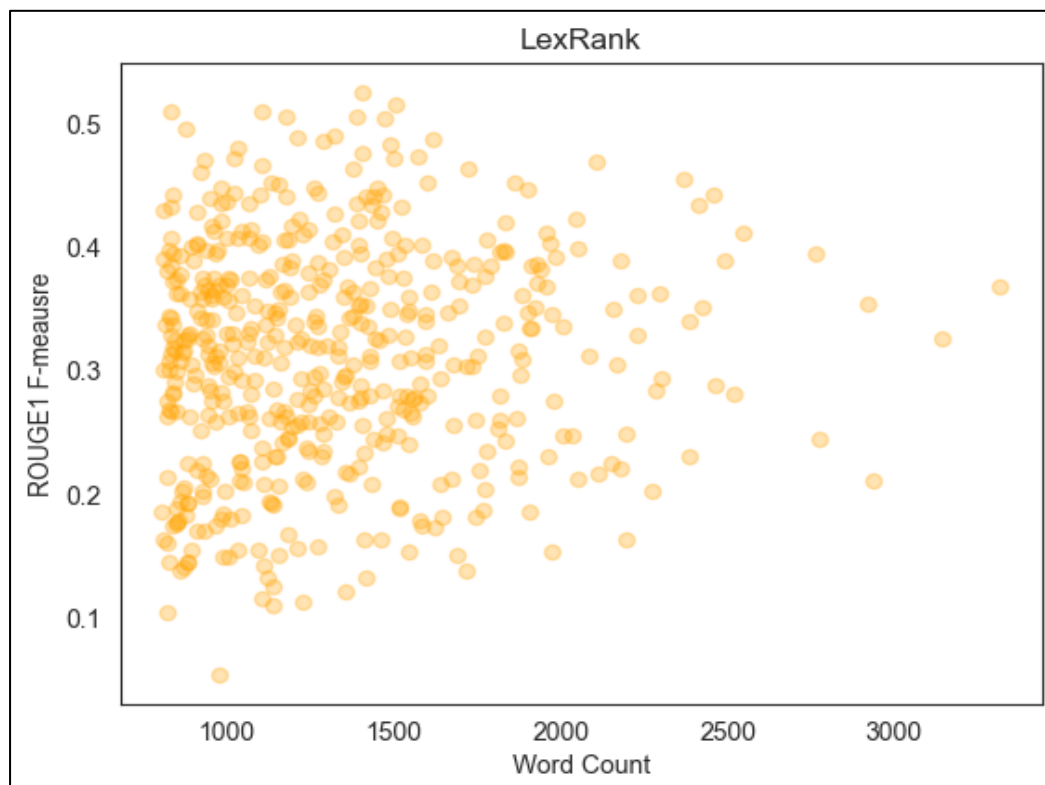


6.5.6 Scatter Plot - Word count vs ROUGE-1

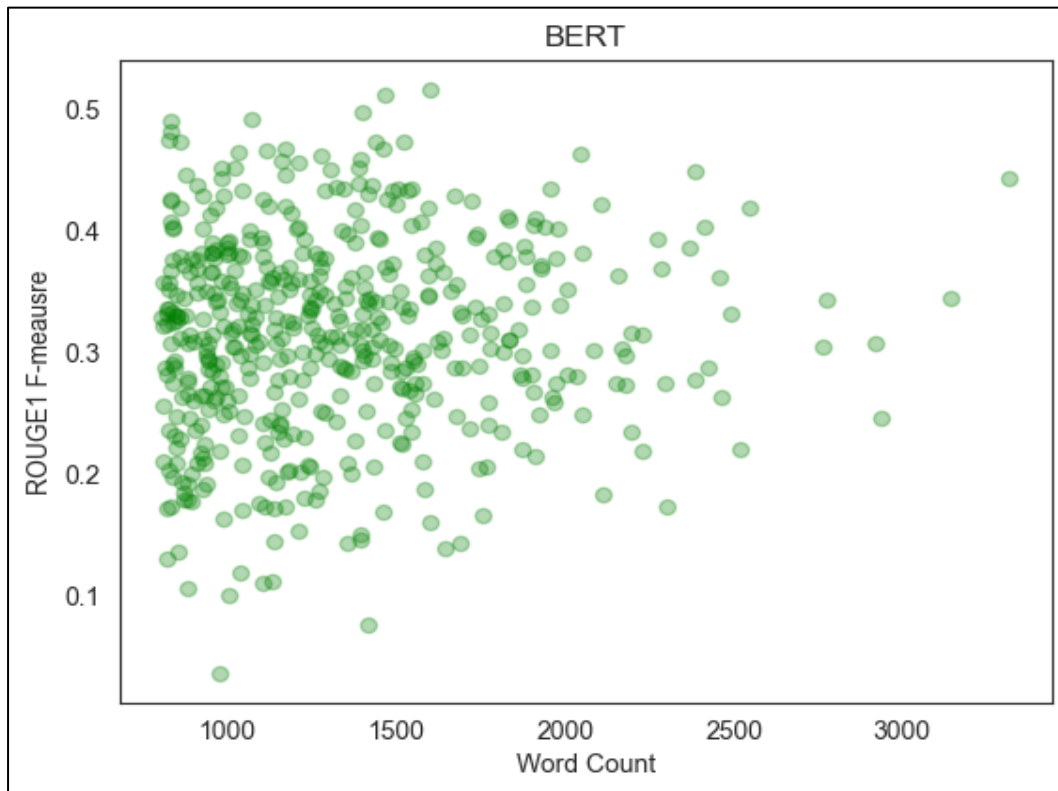
1. TF-IDF



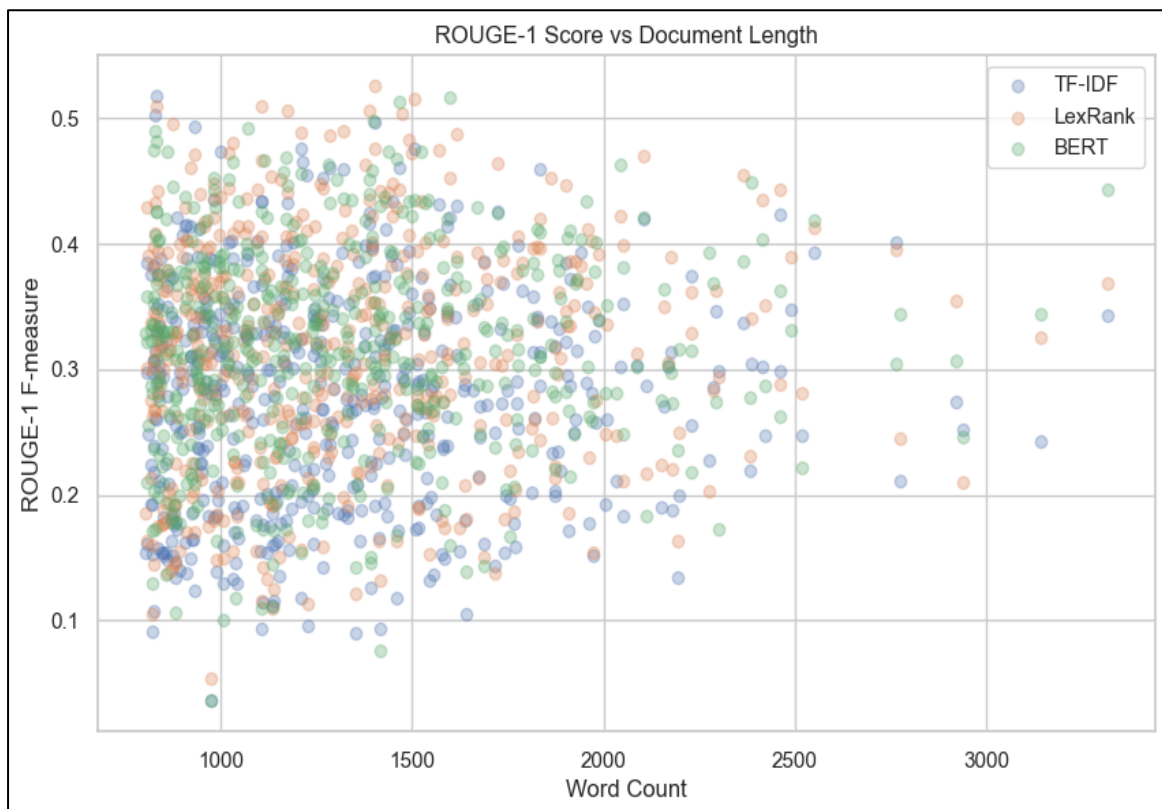
2. LexRank



3. SBERT



4. All Models

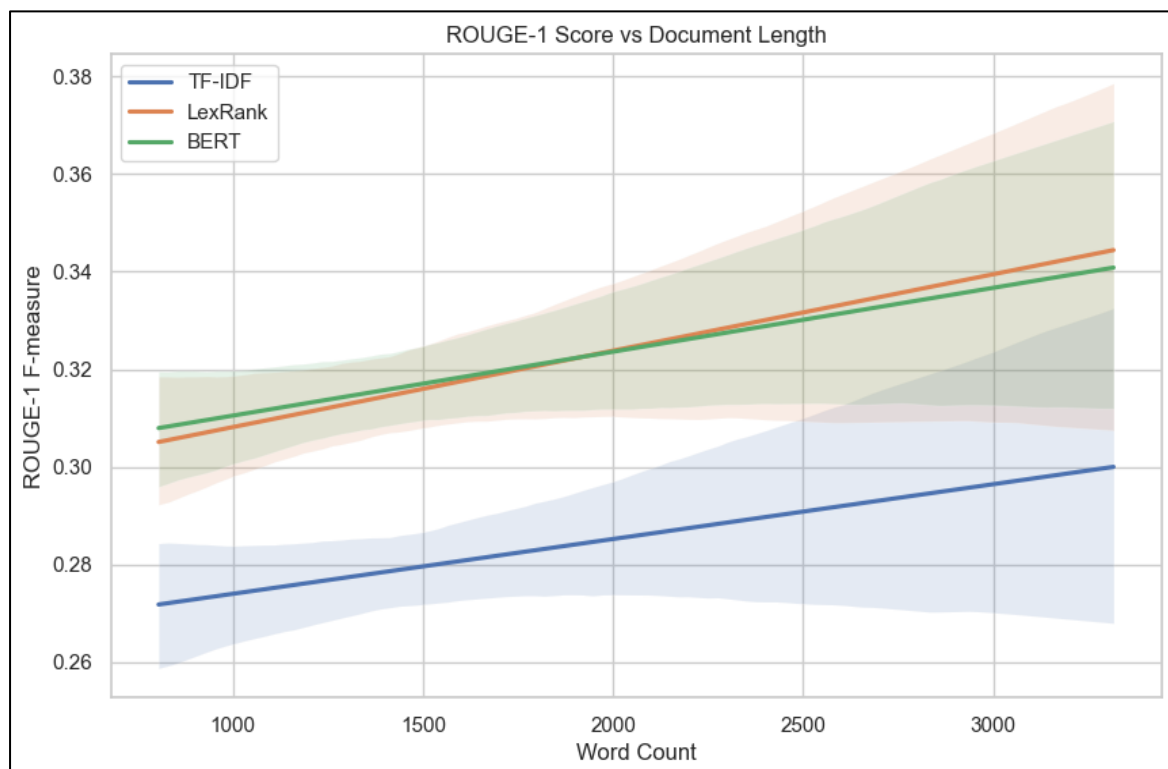


Scatter Plot shows the raw distribution of scores across document lengths. It shows how performance changes as articles get longer.

The plot is a bit hard to read because of the overlapping points. It shows **no clear degradation with increasing document length**. It demonstrates high variance and overlaps between models.

6.5.7 Regression Plot - Word count vs ROUGE-1

It shows the trend line for each model, making relationship with document length clearer.



Regression plot does not plot the actual ROUGE scores. It plots a linear regression trend line. It shows best fit trend line for each method.

The mean ROUGE-1 scores (BERT: 0.309, LexRank: 0.278, TF-IDF: 0.250) tell you the absolute performance - BERT performs better overall. The regression plot shows how scores change as documents get longer.

The above plot shows that LexRank performed slightly better than SBERT as document length increases, which appears contrary to the mean scores where SBERT outperformed LexRank overall (0.309 vs 0.278). However, regression lines are sensitive to distribution of document lengths, variance, and outliers. Earlier, we saw box plot showed that SBERT has

few outliers and scatter plot showed high variance between models. This can make the trend line appear above another line in certain regions even if its overall average is lower.

Scatter plot suggests and Regression plot further supports that there's no clear performance degradation as document length increases, as slope is similar across all three models. This is contrary to what general consensus regarding long document text summarization is, that is as document length increases, the traditional statistical method's performance starts degrading and BERT shows increasing performance.

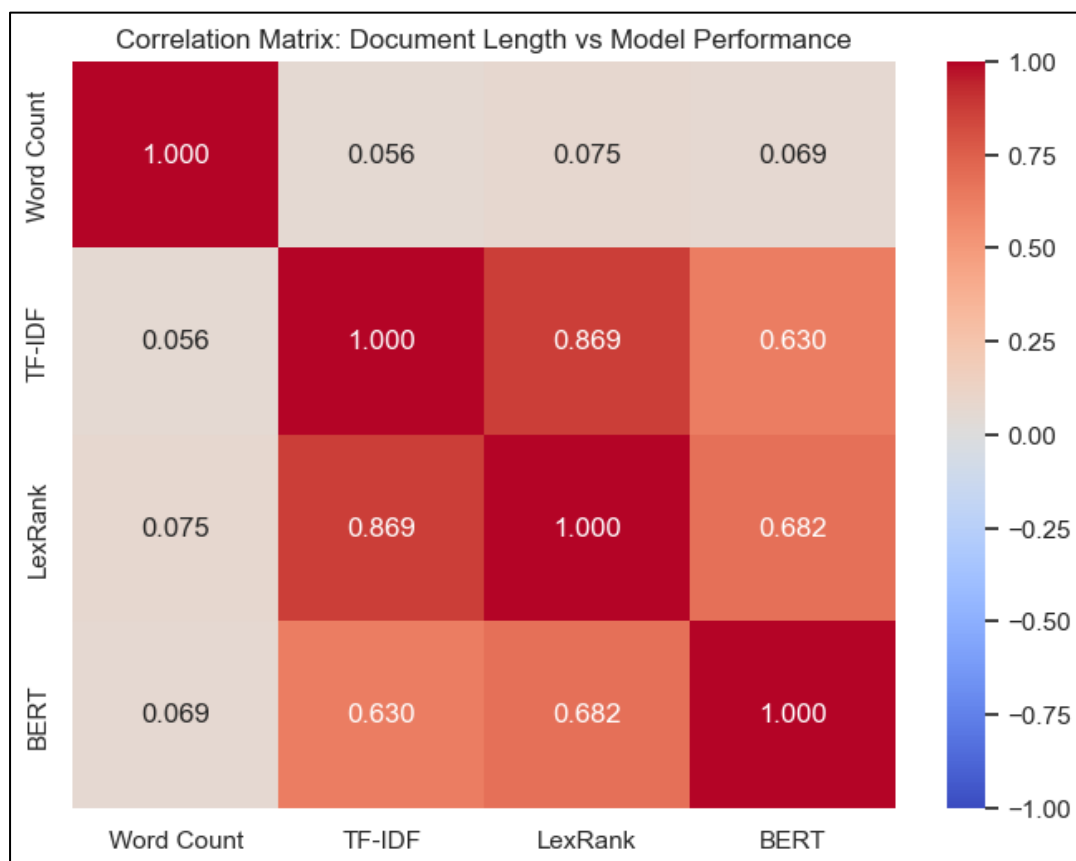
This could be due to couple of reasons:

1. We are already in long document range (800+ words)
2. The high variance we saw in the scatter plot suggests that the regression line has low confidence i.e. it is not a strong fit. It is possible that regression plot might be oversimplifying this as the trend line is very subtle.

This suggests that among long documents (800+ words), there's no evidence of performance degradation with increasing document length.

6.5.8 Heat Map – Correlation Matrix

The Heat map visually shows the correlation matrix of Doc length vs Model Performance. Darker shades indicate stronger positive correlations, whereas lighter shades indicate weaker correlations.



Correlation analysis revealed high correlation between the models (TF-IDF and LexRank: 0.869 – strongly correlated, TF-IDF and BERT: 0.630 — moderately correlated, LexRank and BERT: 0.682 — moderately correlated).

This suggests TF-IDF and LexRank tend to perform similarly on the same documents. This is expected because both approaches rely primarily on lexical information for sentence importance estimation. In contrast, BERT’s moderate correlation with TF-IDF and LexRank suggest that the SBERT-based method exhibits a somewhat different sentence selection behavior due to its use of semantic sentence embeddings.

Pearson correlation between document length and ROUGE-1 F-measure was weak for all three models (TF-IDF: $r=0.056$, LexRank: $r=0.075$, BERT: $r=0.069$), suggesting that within long document range - 800+ words, word count exhibited a negligible linear correlation with ROUGE-1 scores for all summarization methods.

These findings suggest that, once documents exceed the long-document threshold (800+ words) used in this study, variations in document length appear to have minimal influence on the performance of summarization model. However, the models did not perform equivalently, as BERT consistently achieved higher average ROUGE scores than both TF-IDF and LexRank. Therefore, while no evidence of performance degradation with increasing document length was observed within the investigated range, the choice of summarization method remained an important determinant of summary quality.

Since the present study focused exclusively on long documents (800+ words), further research is needed to determine whether similar trends hold for short and medium-length documents.

(Note: Strong correlation - [0.7 to 1.0 or -0.7 to -1.0], Moderate correlation - [0.3 to 0.7 or -0.3 to -0.7], Weak correlation – [0.0 to 0.3 or 0.0 to -0.3])

7. CONCLUSION:

This study presented a comparative analysis of three extractive text summarization techniques - TF-IDF, LexRank, and a cluster-based extractive summarization approach using Sentence-BERT (SBERT) embeddings - for summarizing long documents. The WikiHow dataset was selected as the benchmark dataset, and 500 articles containing more than 800 words were used for evaluation. The generated summaries were assessed using ROUGE metrics, redundancy analysis, and coverage.

The experimental results demonstrated that the SBERT-based approach achieved the highest overall summarization performance with a ROUGE-1 score of 0.309 and a ROUGE-L score of 0.166, indicating its superior ability to identify semantically important sentences in long documents. LexRank consistently outperformed TF-IDF and obtained the highest ROUGE-2 score (0.078). TF-IDF produced the lowest performance among the three methods.

A notable finding of this study was the negligible positive correlation ($r \approx 0.05-0.07$) observed between document length and ROUGE-1 scores across all three models. This suggests that within the long-document range (800+ words), performance does not significantly degrade with increasing document length.

The study also highlighted a trade-off between coverage and redundancy. TF-IDF achieved the highest lexical coverage (0.532) but also exhibited higher redundancy (0.399), while SBERT produced the least redundant summaries (0.325) at the cost of lower lexical coverage (0.370). This suggests that lexical overlap metrics such as ROUGE may not fully capture the semantic quality of SBERT-generated summaries, pointing to a limitation of current evaluation approaches.

Although the cluster-based SBERT approach consistently achieved the best overall performance, the improvement over LexRank was relatively modest. These findings indicate that extractive summarization of long documents remains a challenging task. Overall, the study demonstrates that SBERT embedding-based summarization methods offer measurable improvements over traditional statistical and graph-based techniques while maintaining low redundancy and better semantic representation of document content.

8. FUTURE ENHANCEMENT:

Several directions could extend this work. Future studies may extend evaluation on domain-specific long-document datasets such as PubMed scientific articles or legal documents, which would provide insights into how extractive methods generalize across different document types and writing styles.

Since this study focused exclusively on the long-document range (800+ words), future work could extend the analysis to include short and medium-length documents to examine whether performance degradation occurs in those ranges - a trend not observable within this study's scope.

Furthermore, if degradation is observed, further research could identify the specific word count threshold at which performance stabilizes. Such analysis would provide a clearer understanding of the document length boundaries within which each extractive method operates effectively.

Finally, semantic coverage metrics could be used to replace or complement ROUGE-based coverage metric used in this study. This will provide a more accurate reflection of summary quality for embedding-based methods like SBERT.

REFERENCES:

- [1] I. Mani and M. T. Maybury, *Advances in Automatic Text Summarization*. MIT Press, 1999.
- [2] A. K. S. J. Rajput, "Extractive Text Summarization Using Sentence Ranking," *IEEE Xplore*, 2019. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8817040>
- [3] J. Zhang et al., "A Comprehensive Survey on Process-Oriented Automatic Text Summarization with Exploration of LLM-Based Methods," *arXiv preprint arXiv:2403.02901*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.02901>
- [4] Y. Liu and M. Lapata, "Text Summarization with Pretrained Encoders," *arXiv preprint arXiv:1908.08345*, 2019. (Note: Also known as the BertSum framework). [Online]. Available: <https://arxiv.org/abs/1908.08345>
- [5] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1-67, 2020. [Online]. Available: <https://www.jmlr.org/papers/volume21/20-074/20-074.pdf>
- [6] S. El-Kassas et al., "A weighted word embedding based approach for extractive text summarization," *Expert Systems with Applications*, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0957417421012264>
- [7] L. C. S. Silva and R. R. Sampaio, "Using Luhn's Automatic Abstract Method to Create Graphs of Words for Document Visualization," *Social Networking*, vol. 3, no. 2, 2014. [Online]. Available: <https://www.scirp.org/journal/paperinformation?paperid=43321>
- [8] H. P. Luhn, "The Automatic Creation of Literature Abstracts," *IBM Journal of Research and Development*, vol. 2, no. 2, pp. 159-165, 1958. [Online]. Available: <https://calv.info/luhns-auto-abstract-algorithm>
- [9] R. S. S. S. Prasad and S. Kumar, "Extractive Text Summarization from Web pages using Selenium and TF-IDF algorithm," *IEEE Xplore*, 2020. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9142981>

- [10] Y. Erraji, "Understanding TextRank: A Deep Dive into Graph-Based Text Summarization and Keyword Extraction," Medium, 2023. [Online]. Available: <https://medium.com/@yassineerraji/understanding-textrank-a-deep-dive-into-graph-based-text-summarization-and-keyword-extraction-905d1fb5d266>
- [11] G. Erkan and D. R. Radev, "LexRank: Graph-based lexical centrality as salience in text summarization," *Journal of Artificial Intelligence Research*, vol. 22, pp. 457–479, 2004.
- [12] A. K. Yadav, Ranvijay, R. S. Yadav, and A. K. Maurya, "Graph-based extractive text summarization based on single document," *Multimedia Tools and Applications*, vol. 83, pp. 18987–19013, 2024. [Online]. Available: <https://doi.org/10.1007/s11042-023-16199-8>
- [13] V. Dalal and L. G. Malik, "Semantic Graph Based Automatic Text Summarization for Hindi Documents Using Particle Swarm Optimization," in *Proceedings of the 5th International Conference on Advanced Computing, Networking and Informatics (ICACNI)*, 2017. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-63645-0_31
- [14] S. Ullah and A. B. M. Alim Al Islam, "A framework for extractive text summarization using semantic graph based approach," in *Proceedings of the 16th International Conference on Distributed Computing and Internet Technology (ICDCIT)*, 2020. [Online]. Available: <https://dl.acm.org/doi/10.1145/3362966.3362971>
- [15] M. Afsharizadeh, H. Ebrahimpour-Komleh, and A. Bagheri, "A new graph-based extractive text summarization using keywords or topic modeling," *Applied Intelligence*, vol. 51, pp. 1092–1111, 2021. [Online]. Available: <https://doi.org/10.1007/s12652-020-02591-x>
- [16] A. Joshi, E. Fidalgo, E. Alegre, and L. Fernández-Robles, "SummCoder: An unsupervised framework for extractive text summarization based on deep auto-encoders," *Expert Systems with Applications*, vol. 129, pp. 200–215, 2019. [Online]. Available: <https://doi.org/10.1016/j.eswa.2019.03.045>
- [17] Y. Liu, "Fine-tune BERT for Extractive Summarization," *arXiv preprint arXiv:1903.10318*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.10318>
- [18] S. Miller, "Performance Study on Extractive Text Summarization Using BERT Models," *Information*, vol. 13, no. 2, p. 67, 2022. [Online]. Available: <https://www.mdpi.com/2078-2489/13/2/67>

[19] M. Azam, S. Khalid, S. Almutairi, and H. S. M. Bilal, "Summarization of scholarly articles using BERT and BiGRU: Deep learning-based extractive approach," *Journal of King Saud University - Computer and Information Sciences*, vol. 35, no. 9, 2023. [Online].

Available: <https://doi.org/10.1016/j.jksuci.2023.101739>

[20] Q. Xie, J. A. Bishop, P. Tiwari, and S. Ananiadou, "Pre-trained language models with domain knowledge for biomedical extractive summarization," *Knowledge-Based Systems*, vol. 252, p. 109460, 2022. [Online]. Available: <https://doi.org/10.1016/j.knosys.2022.109460>

[21] W. X. Zhao et al., "A Systematic Survey of Text Summarization: From Statistical Methods to Large Language Models," *ACM Computing Surveys*, 2024. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3731445>

[22] M. Koupae and W. Y. Wang, "WikiHow: A Large Scale Text Summarization Dataset," *arXiv preprint arXiv:1810.09305*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.09305>

[23] M. Koupae and W. Y. Wang, "WikiHow: A Large Scale Text Summarization Dataset," *arXiv preprint arXiv:1810.09305*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.09305>

[24] M. Azam et al., "Current Trends and Advances in Extractive Text Summarization: A Comprehensive Review," *IEEE Access*, vol. 12, pp. 11096-11118, 2024. [Online]. Available: <https://doi.org/10.1109/ACCESS.2024.10872906>

[25] P. Gupta, S. Nigam, and R. Singh, "A Ranking based Language Model for Automatic Extractive Text Summarization," in *2022 First International Conference on Artificial Intelligence Trends and Pattern Recognition (ICAITPR)*, Hyderabad, India, 2022, pp. 1-5. [Online]. Available: <https://doi.org/10.1109/ICAITPR51569.2022.9844187>

[26] T. Vo, "SGCSumm: An extractive multi-document summarization method based on pre-trained language model, submodularity, and graph convolutional neural networks," *Expert Systems with Applications*, vol. 211, p. 119308, 2023. [Online]. Available: <https://doi.org/10.1016/j.eswa.2022.119308>

[27] A. Joshi, E. Fidalgo, E. Alegre, and L. Fernández-Robles, "DeepSumm: Exploiting topic models and sequence to sequence networks for extractive text summarization," *Expert Systems with Applications*, vol. 211, p. 118442, 2023. [Online]. Available: <https://doi.org/10.1016/j.eswa.2022.118442>

APPENDIX – PROGRAM CODE:**A. Creating Virtual Environment:**

→Install Python using Python Install Manager:

```
py install 3.15
```

→Check all the installed python version:

```
py list
```

→Create Virtual environment:

```
py -3.11 -m venv venv-name
```

→Activate Virtual environment:

```
venv-name\Scripts\activate
```

B. Notebook 1: TF-IDF

```
import nltk
import sklearn
import sentence_transformers
import rouge_score

df = pd.read_csv('wikihowAll.csv')

df.dropna(inplace=True)

df['word count'] = df['text'].apply(lambda x: len(x.split()))

len(df[df['word count'] > 800])

df_long = df[df['word count'] > 800].reset_index(drop=True)

sample = df_long.sample(n=500, random_state=1)

print(sample['text'].iloc[0][:10000])
```

#Preprocessing Sample Dataset:

```
sample['text'] = sample['text'].str.replace('\n', ' ', regex=True)
sample['text'] = sample['text'].str.replace('\n', ' ', regex=True)
sample['text'] = sample['text'].str.strip()
```

```

print(sample['headline'].iloc[0])

sample['headline'] = sample['headline'].str.replace('\n', ' ', regex=True)
sample['headline'] = sample['headline'].str.replace('.', ' ', regex=True)
sample['headline'] = sample['headline'].str.strip()

print(sample['headline'].iloc[0])

#Tokenization:
import nltk
nltk.download('punkt') #NLTK's Tokenizer.
nltk.download('stopwords')
nltk.download('punkt_tab')

sentences = nltk.sent_tokenize(sample['text'].iloc[0])

sample['sentences'] = sample['text'].apply(lambda x :
nltk.sent_tokenize(x))

#TF-IDF Summarizer:

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(sample['sentences'].iloc[0])

vectorizer.get_feature_names_out()

print(X.shape)

X.toarray()

sentence_scores = X.toarray().sum(axis=1)

sentence_scores.shape

top_indices = sentence_scores.argsort()[::-1][:5]

sorted_indices = sorted(top_indices)

sentences = sample['sentences'].iloc[0]
summary = ' '.join(sentences[i] for i in sorted_indices)

print(summary)

#TD-IDF Summarizer Function:
def tfidf_summarizer(sentences):
    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(sentences)
    sentence_scores = X.toarray().sum(axis=1)

```

```

top_indices = sentence_scores.argsort()[::-1][:5]
sorted_indices = sorted(top_indices)
summary = ' '.join(sentences[i] for i in sorted_indices)
return summary

sample['tfidf_summary'] = sample['sentences'].apply(tfidf_summarizer)

# Evaluation:

from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)

scores = scorer.score(sample['headline'].iloc[0],
sample['tfidf_summary'].iloc[0])

sample['tfidf_score'] = sample.apply(lambda x: scorer.score(x['headline'],
x['tfidf_summary']), axis=1)

sample['tfidf_score'].iloc[0]['rouge1'].fmeasure

sample['rouge1_f'] = sample['tfidf_score'].apply(lambda x :
x['rouge1'].fmeasure)
sample['rouge2_f'] = sample['tfidf_score'].apply(lambda x :
x['rouge2'].fmeasure)
sample['rougeL_f'] = sample['tfidf_score'].apply(lambda x :
x['rougeL'].fmeasure)

print(sample[['rouge1_f', 'rouge2_f', 'rougeL_f']].mean())

```

C. Notebook 2: LexRank

```

import sumy
import pandas as pd
import rouge_score

sample = pd.read_csv('sample.csv')

from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.lex_rank import LexRankSummarizer

parser = PlaintextParser.from_string(sample['text'].iloc[0],
Tokenizer("english"))

```

```

parser.document.sentences

summarizer = LexRankSummarizer()

summary_sentences = summarizer(parser.document, 5)

summary = ' '.join(str(sentences) for sentences in summary_sentences)

# LexRank Summarizer Function:

def lexranks_summarizer(text, num_sentences=5):
    parser = PlaintextParser.from_string(text, Tokenizer("english"))
    summarizer = LexRankSummarizer()
    summary_sentences = summarizer(parser.document, num_sentences)
    summary = ' '.join(str(sentences) for sentences in summary_sentences)
    return summary

sample['lexrank_summary'] = sample['text'].apply(lexranks_summarizer)

# Evaluation:

from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)

scores = scorer.score(sample['headline'].iloc[0],
sample['lexrank_summary'].iloc[0])

sample['lexrank_score'] = sample.apply(lambda x:
scorer.score(x['headline'], x['lexrank_summary']), axis=1)

sample['lexrank_score'].iloc[0]['rouge1'].fmeasure

sample['lexrank_rouge1_f'] = sample['lexrank_score'].apply(lambda x :
x['rouge1'].fmeasure)
sample['lexrank_rouge2_f'] = sample['lexrank_score'].apply(lambda x :
x['rouge2'].fmeasure)
sample['lexrank_rougeL_f'] = sample['lexrank_score'].apply(lambda x :
x['rougeL'].fmeasure)

print(sample[['lexrank_rouge1_f', 'lexrank_rouge2_f',
'lexrank_rougeL_f']].mean())

```

D. Notebook 3: SBERT

```

from sentence_transformers import SentenceTransformer
import pandas as pd

# Loading the Model:
model = SentenceTransformer('all-MiniLM-L6-v2')

sample = pd.read_csv('sample1.csv')

import ast

sample['sentences'] = sample['sentences'].apply(ast.literal_eval)
#changes data type of sentences from 'str' to 'object'

# Training the Model:
embeddings = model.encode(sentences)
embeddings.shape

# Intializing KMeans:
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=5, random_state=0,
n_init='auto').fit(embeddings)

kmeans.cluster_centers_

from sklearn.metrics import pairwise_distances_argmin

closest_indices = pairwise_distances_argmin(kmeans.cluster_centers_,
embeddings)

sorted_indices = sorted(closest_indices)

summary = ' '.join(sentences[i] for i in sorted_indices)

# BERT Summarizer Function:
def bert_summarizer(sentences):
    embeddings = model.encode(sentences)
    if len(sentences) >= 5:
        cluster = 5
    else:
        cluster = len(sentences)
    kmeans = KMeans(n_clusters=cluster, random_state=0,
n_init='auto').fit(embeddings)
    closest_indices = pairwise_distances_argmin(kmeans.cluster_centers_,
embeddings)
    sorted_indices = sorted(closest_indices)
    summary = ' '.join(sentences[i] for i in sorted_indices)

```

```

    return summary

sample['bert_summary'] = sample['sentences'].apply(bert_summarizer)

# Evaluation:
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)

scores = scorer.score(sample['headline'].iloc[0],
sample['bert_summary'].iloc[0])

sample['bert_score'] = sample.apply(lambda x: scorer.score(x['headline'],
x['bert_summary']), axis=1)

sample['bert_score'].iloc[0]['rouge1'].fmeasure

print(sample[['bert_rouge1_f', 'bert_rouge2_f', 'bert_rougeL_f']].mean())

# Content Coverage:

->BERT Coverage
sample['bert_coverage'] = sample['bert_score'].apply(lambda x :
x['rouge1'].recall)

->TFIDF Coverage
sample['tfidf_score'] = sample.apply(lambda x :
scorer.score(x['headline'], x['tfidf_summary']), axis=1)

sample['tfidf_coverage'] = sample['tfidf_score'].apply(lambda x :
x['rouge1'].recall)

->LexRank Coverage
sample['lexrank_score'] = sample.apply(lambda x :
scorer.score(x['headline'], x['lexrank_summary']), axis=1)

sample['lexrank_coverage'] = sample['lexrank_score'].apply(lambda x :
x['rouge1'].recall)

sample[['bert_coverage', 'tfidf_coverage', 'lexrank_coverage']].mean()
#Average Content coverage scores.

```

```

# Redundancy:
import nltk
nltk.download('punkt') #NLTK's Tokenizer.
nltk.download('stopwords')
nltk.download('punkt_tab')

import numpy as np

tokenized_summary = nltk.sent_tokenize(summary) # Summary is tokenized to
avoid hitting the SBERT's 512 token limit.

embeddings1 = model.encode(tokenized_summary)

from sklearn.metrics.pairwise import cosine_similarity

x = cosine_similarity(embeddings1) #Computes pairwise cosine similarity
between summary sentences using their embeddings.
print('Shape:\n', x.shape)
print('Matrix:\n', x)

upper = np.triu(x, k=1)

upper[upper > 0] #upper[upper > 0] filters out those zeros and computes
the mean of only the real similarity values.

redundancy = upper[upper > 0].mean()

#Redundancy Function

def calc_redundancy(summary):
    tokenized_summary = nltk.sent_tokenize(summary)
    embeddings = model.encode(tokenized_summary)
    x = cosine_similarity(embeddings)
    upper = np.triu(x, k=1)
    redundancy = upper[upper > 0].mean()
    return redundancy

#TFIDF Redundancy
sample['tfidf_redundancy'] =
sample['tfidf_summary'].apply(calc_redundancy)

#LexRank Redundancy
sample['lexrank_redundancy'] =
sample['lexrank_summary'].apply(calc_redundancy)

#BERT Redundancy
sample['bert_redundancy'] = sample['bert_summary'].apply(calc_redundancy)

sample[['tfidf_redundancy', 'lexrank_redundancy',
'bert_redundancy']].mean()

sample.to_pickle('final_df.pkl') #Saving Final Dataframe as pickle file.

```

E. Notebook 4: Visualization

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sample = pd.read_pickle('final_df.pkl')
```

```
sns.set_theme(style="whitegrid")
```

1. ROUGE Comparison (Bar Chart)

```
results = pd.DataFrame({
    'Model': ['TF-IDF', 'TF-IDF', 'TF-IDF', 'LexRank', 'LexRank',
    'LexRank', 'BERT', 'BERT', 'BERT'],
    'Metric': ['ROUGE-1', 'ROUGE-2', 'ROUGE-L', 'ROUGE-1', 'ROUGE-2',
    'ROUGE-L', 'ROUGE-1', 'ROUGE-2', 'ROUGE-L'],
    'Score': [0.278, 0.066, 0.148, 0.313, 0.081, 0.170, 0.315, 0.072,
    0.174]
})
```

```
plt.figure(figsize=(8, 5))
sns.barplot(data=results, x='Metric', y='Score', hue='Model')
plt.title('ROUGE Comparison')
```

```
plt.figure(figsize=(8, 5))
sns.lineplot(data=results, x='Metric', y='Score', hue='Model', marker='o',
linewidth=2.5, markersize=8)
plt.title('ROUGE Comparison')
```

2.Redundancy & Content Coverage Comparison (Bar Chart)

```
results2 = pd.DataFrame({
    'Model': ['TFIDF', 'TFIDF', 'LexRank', 'LexRank', 'BERT', 'BERT'],
    'Metric': ['Coverage', 'Redundancy', 'Coverage', 'Redundancy',
    'Coverage', 'Redundancy'],
    'Score': [0.532, 0.399, 0.487, 0.442, 0.370, 0.325]
})
```

```
plt.figure(figsize=(8, 5))
sns.barplot(data=results2, x='Metric', y='Score', hue='Model')
plt.title('Redundancy & Content Coverage Comparison')
```

3.Content Coverage vs Redundancy

```

scatter_data = pd.DataFrame({
    'Model': ['TF-IDF', 'LexRank', 'BERT'],
    'Coverage': [0.532, 0.487, 0.370],
    'Redundancy': [0.399, 0.442, 0.325]
})
scatter_data

plt.figure(figsize=(8, 5))
sns.scatterplot(data=scatter_data, x='Coverage', y='Redundancy',
hue='Model', s=100)
plt.title('Content Coverage vs Redundancy')

line_data = scatter_data.melt(id_vars='Model', var_name='Metric',
value_name='score')
line_data

plt.figure(figsize=(8, 5))
sns.lineplot(data=line_data, x='Model', y='score', hue='Metric',
marker='o', linewidth=2.5, markersize=8)
plt.title('Coverage vs Redundancy')

```

4.Box Plot (ROUGE-1 Distribution)

```

rouge1_data = sample[['rouge1_f', 'lexrank_rouge1_f',
'bert_rouge1_f']].melt()
#.melt() - unpivots the dataframe i.e. from many columns to many rows.
rouge1_data

plt.figure(figsize=(8, 5))
sns.boxplot(data=rouge1_data, x='variable', y='value', hue='variable')
plt.xticks(ticks=[0, 1, 2], labels=['TFIDF', 'LexRank', 'BERT (SBERT)'])
plt.xlabel('-----Models-----')
plt.ylabel('-----Score-----')
plt.title('ROUGE-1 Distribution')
plt.show()

```

5.Scatter Plot (Word count vs ROUGE-1)

```

plt.scatter(sample['word count'], sample['rouge1_f'], alpha=0.3,
label='TF-IDF')
plt.xlabel('Word Count')
plt.ylabel('ROUGE1 F-measre')
plt.title('TF-IDF')

```

```
plt.scatter(sample['word count'], sample['lexrank_rouge1_f'], alpha=0.3,
label='LexRank', color='orange')
plt.xlabel('Word Count')
plt.ylabel('ROUGE1 F-measure')
plt.title('LexRank')
```

```
plt.scatter(sample['word count'], sample['bert_rouge1_f'], alpha=0.3,
label='BERT', color = 'green')
plt.xlabel('Word Count')
plt.ylabel('ROUGE1 F-measure')
plt.title('BERT')
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(sample['word count'], sample['rouge1_f'], alpha=0.3,
label='TF-IDF')
plt.scatter(sample['word count'], sample['lexrank_rouge1_f'], alpha=0.3,
label='LexRank')
plt.scatter(sample['word count'], sample['bert_rouge1_f'], alpha=0.3,
label='BERT')
```

```
plt.xlabel('Word Count')
plt.ylabel('ROUGE-1 F-measure')
plt.title('ROUGE-1 Score vs Document Length')
plt.legend()
plt.show()
```

6. Regression Plot (Word count vs ROUGE-1)

```
plt.figure(figsize=(10, 6))

for col, label in zip(['rouge1_f', 'lexrank_rouge1_f', 'bert_rouge1_f'],
                      ['TF-IDF', 'LexRank', 'BERT']):
    sns.regplot(data=sample, x='word count', y=col, scatter=False,
label=label) #ci=None)

plt.xlabel('Word Count')
plt.ylabel('ROUGE-1 F-measure')
plt.title('ROUGE-1 Score vs Document Length')
plt.legend()
plt.show()
```

7. Correlation Matrix

```
sample[['word count',
        'rouge1_f',
        'lexrank_rouge1_f',
        'bert_rouge1_f']].corr()
```

```
plt.figure(figsize=(8, 6))
clean_labels = ['Word Count', 'TF-IDF', 'LexRank', 'BERT']
# annot=True puts the numbers in the boxes, cmap sets a nice scientific
color profile
sns.heatmap(sample[['word count', 'rouge1_f', 'lexrank_rouge1_f',
'bert_rouge1_f']].corr(),
            annot=True, cmap='coolwarm', vmin=-1, vmax=1,
fmt=".3f", xticklabels=clean_labels,
            yticklabels=clean_labels)

plt.title('Correlation Matrix: Document Length vs Model Performance')
plt.show()
```